

Contributor: Shalini

Role

Voice Agent, Video Creation & System Integration

Tech Stack

FastAPI Python gTTS FFmpeg SRT (burned-in subtitles) React + TypeScript REST API (v1)

1. Voice Agent

- Built the voice narration layer feeding into the Media Decision Engine (Stage 2).
- Used **gTTS** to generate spoken narration for each video scene.
- Output audio is synced to on-screen captions, which are burned into the final video as **SRT subtitles** via FFmpeg.

How it works: script text (from the Content Agent) → gTTS converts it to speech per scene → FFmpeg assembles the audio track with the video frames and burns in the caption track, so the final MP4 ships with narration + subtitles already embedded — no separate subtitle file needed at playback time.

2. Video Creation

- Built the multi-scene video assembly pipeline: one generated/fetched image per scene, stitched together with narration and captions into a single MP4.
- Handled the video cache check — if a video already exists for a given brief, it's reused instead of regenerated (avoids redundant, slow re-generation).
- Handled the fallback path when no AI image provider is available: a clean "Branded fallback" placeholder card is used per scene instead of failing the whole video, and it's explicitly tagged so it's never mistaken for an AI-generated visual.

3. Integration

- Connected the **React + TypeScript frontend (port 5173)** to the **consolidated FastAPI backend (port 8000)** over REST `v1` endpoints.
- Wired the voice + video output into the shared pipeline so it flows correctly from Content Agent → Media Decision Engine → Compliance Gate → Review/Publish, rather than living as a disconnected module.
- Responsible for making sure all “12 Managed Screens” (Dashboard, Content Studio, Review Center, Audit Log, etc.) talk to the same backend state.

Notes

Fill in specific file paths / module names (e.g. `media_engine.py` , `voice_agent.py`) and any FFmpeg command flags used, before committing to GitHub.